

```
satya@Elegance: ~/Codes
[Satya]~/Codes
gcc -Wall program_1.c
program_1.c: In function 'main':
program_1.c:6:4: warning: operation on 'a' may be undefined [-Wsequence-point]
    a = a++ * a++;
    ~~~~~
program_1.c:6:4: warning: operation on 'a' may be undefined [-Wsequence-point]
[Satya]~/Codes
```

Figure 1: Output of the code shown in program-1, compiled using gcc compiler

```
satya@Elegance: ~/Codes
[Satya]~/Codes
./a.out
The value of a: 110
[Satya]~/Codes
```

Figure 1a: Output of the code shown in program-1, run under an Intel based Linux platform

Case study 1

Consider code snippet-1, for which the complete code is given in the program-1 below:

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int a = 10;
6     a = a++ * a++;
7     printf("The value of a: %d\n", a);
8
9     return 0;
10 }
```

From this program, in Line 6 it is very clear that the value of 'a' is modified more than once between the previous and the next sequence points. For experimental purposes, I have used two different compilers to compile and run. And the object code has been generated with the following two different compilers.

gcc compiler -- version 4.8.4 (Ubuntu 4.8.4-2ubuntu1~14.04.3)
clang compiler - version 3.8.0-2ubuntu3~trusty4 (tags/RELEASE_380/final)

From the output shown in Figure 1, the gcc compiler generates a warning when compiled with the -Wall option, and the output obtained when this is executed is shown in Figure 1a.

When the same code is compiled using a Clang compiler, the output obtained is shown in Figure 3 and assembly code is shown in Figure 4.

As can be seen from Figures 1 and 3, both the compilers generate the warnings. One can even see that the assembly codes generated from both the compilers are different.

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int a = 10;
6     a = a++ + a++ + a++;
```

```
00000000040052d <main>:
#include <stdio.h>

int main()
{
    40052d: 55                push    %rbp
    40052e: 48 89 e5          mov     %rsp,%rbp
    400531: 48 83 ec 10       sub     $0x10,%rsp
        int a = 10;
    400535: c7 45 fc 0a 00 00 00 movl    $0xa,-0x4(%rbp)
        a = a++ * a++;
    40053c: 8b 55 fc          mov     -0x4(%rbp),%edx
    40053f: 8d 42 01          lea     0x1(%rdx),%eax
    400542: 89 45 fc          mov     %eax,-0x4(%rbp)
    400545: 8b 45 fc          mov     -0x4(%rbp),%eax
    400548: 8d 48 01          lea     0x1(%rax),%ecx
    40054b: 89 4d fc          mov     %ecx,-0x4(%rbp)
    40054e: 0f af c2          imul    %edx,%eax
    400551: 89 45 fc          mov     %eax,-0x4(%rbp)
        printf("The value of a: %d\n", a);
    400554: 8b 45 fc          mov     -0x4(%rbp),%eax
    400557: 89 c6             mov     %eax,%esi
    400559: bf f4 05 40 00    mov     $0x4005f4,%edi
    40055e: b8 00 00 00 00    mov     $0x0,%eax
    400563: e8 a8 fe ff ff    callq   400410 <printf@plt>

        return 0;
    400568: b8 00 00 00 00    mov     $0x0,%eax
}
```

Figure 2: Assembly code generated for program-1, compiled using gcc compiler

```
satya@Elegance: ~/Codes
[Satya]~/Codes
clang program_1.c
program_1.c:6:7: warning: multiple unsequenced modifications to 'a' [-Wunsequenced]
    a = a++ * a++;
    ~~~~~
1 warning generated.
[Satya]~/Codes
```

Figure 3: Output of the code shown in program-1, compiled using the Clang compiler

```
000000000400530 <main>:
#include <stdio.h>

int main()
{
    400530: 55                push    %rbp
    400531: 48 89 e5          mov     %rsp,%rbp
    400534: 48 83 ec 10       sub     $0x10,%rsp
    400538: 48 8d 3c 25 14 06 40 lea     0x400614,%rdi
    40053f: 00
        int a = 10;
    400547: c7 45 f8 0a 00 00 00 movl    $0xa,-0x8(%rbp)
        a = a++ * a++;
    40054e: 8b 45 f8          mov     -0x8(%rbp),%eax
    400551: 89 c1             mov     %eax,%ecx
    400553: 81 c1 01 00 00 00 add     $0x1,%ecx
    400559: 89 4d f8          mov     %ecx,-0x8(%rbp)
    40055c: 8b 4d f8          mov     -0x8(%rbp),%ecx
    40055f: 89 ca             mov     %ecx,%edx
    400561: 81 c2 01 00 00 00 add     $0x1,%edx
    400567: 89 55 f8          mov     %edx,-0x8(%rbp)
    40056a: 0f af c1          imul    %ecx,%eax
    40056d: 89 45 f8          mov     %eax,-0x8(%rbp)
        printf("The value of a: %d\n", a);
    400570: 8b 75 f8          mov     -0x8(%rbp),%esi
    400573: b0 00             mov     $0x0,%al
    400575: e8 96 fe ff ff    callq   400410 <printf@plt>
    40057a: b9 00 00 00 00    mov     $0x0,%ecx

        return 0;
}
```

Figure 4: Assembly code generated for the program-1, compiled using Clang compiler

```
7     printf("The value of a: %d\n", a);
8
9     return 0;
10 }
```

In program-2 above, in Line 6, the value of variable 'a' is being modified more than once between two sequence points, which leads to undefined behaviour. Compile the code under the gcc compiler with the option -Wall and see the warning generated by the compiler. Repeat compiling the same code under different compilers, then run and see the output and conclude it. This is left as an exercise for programmers to perform.